Stakeholder Agent System

Implementation Progress Report

Regis University - MSSE 692: Software Engineering Practicum I

Matt Bass, Anthony Perez, Joseph Turner, Ramesh Kondala

2-15-2026

Executive Summary

During Week 5, the team made substantial progress toward delivering the backend foundation of the Stakeholder Agent System. The majority of the core infrastructure and service-layer components were implemented, tested, and reviewed. This week focused heavily on establishing a stable architectural base, including the Python FastAPI service, data layer integration with PostgreSQL (Neon), AI adapter integration, logging, exception handling middleware, and foundational domain services.

The data layer is approximately 75% complete and represents the primary carryover item into Week 6. While schema modeling and integration testing are largely in place, environment and setup complications temporarily slowed progress. The Agent Service is approximately 90% complete and awaiting final integration with the message service, which has not yet started due to its dependency on the data layer's completion. The FastAPI controller with Neon authentication integration is also in progress (30%) and required additional investigation due to unexpected complexity in dependency injection and authentication wiring.

Despite these delays, several major components are fully implemented and production-structured, including:

- Pydantic domain models
- Project and Persona services
- Llama Lite LLM
- Wide event logging
- Exception middleware
- Python build and dependency infrastructure

Testing and quality controls are strong for this stage of development. The service and data layers currently report 100% unit test coverage with edge cases included, and the team is consistently using pull requests with automated linting and peer review. Code quality is enforced through Ruff and pre-commit hooks, and type safety is validated via mypy. While CI/CD is not yet fully automated, the groundwork is in place and will be expanded in Week 6.

Overall, the team completed approximately 85% of planned Week 5 backend implementation work. Some effort shifted toward resolving infrastructure and authentication integration issues, but this investment strengthened architectural understanding and reduced long-term risk. The remaining Week 5 carryover items (data layer finalization and API controller completion) are expected to be resolved at the very beginning of Week 6.

The project remains on track for Week 6 goals. No confirmed scope reductions are required at this time. However, the sentiment service may be deferred from the MVP if integration complexity increases. All other planned backend components are expected to be completed next week, followed by API-level integration testing and CI/CD pipeline setup.

In summary, Week 5 was productive and foundational. While minor technical complications introduced short-term delays, the backend architecture is now structurally sound, tested, and positioned for full integration and deployment preparation in Week 6.

Infrastructure & Development Environment

Version control setup and branching strategy

Backend Infrastructure

- Backend is a Python FastAPI service (fastapi[standard])
- Async DB drivers (asyncpg, psycopg) and SQLAlchemy ORM, targeting a Neon PostgreSQL. Dependencies: pyproject.toml.
- AI integration uses pydantic-ai-slim[openai] plus env-configured provider settings.
- Config is driven by .env environment settings.
- DB Migrations through Alembic.

Backend Development Environment

- Python 3.13 (.python-version, requires-python).
- Dependency management via uv (uv.lock in place).
- Pre-commit hooks configured for Ruff lint/format (.pre-commit-config.yaml).
- Dev tooling in pyproject.toml: pytest, pytest-cov, mypy, ruff.
- Local configuration via .env
- Runtime environment managed by uvicorn.

CI/CD status

At this stage, CI is partially implemented and focused on code quality validation.

Current Status

- Pre-commit hooks enforce:
  - Ruff linting
  - Ruff formatting
- Unit tests can be executed locally using pytest.
- Coverage reports available via pytest-cov.

What is Automated

- Linting and formatting checks (locally enforced via pre-commit).

What is Not Yet Automated

- No automated deployment pipeline.
- No automated test execution on pull requests.
- No staging or production deployment environment.

Development environment

Backend

Development environment is a local Python 3.13 setup managed with uv and .venv. Dependencies and dev tools are declared in pyproject.toml and locked in uv.lock. Code quality uses Ruff via pre-commit hooks (.pre-commit-config.yaml), plus pytest/pytest-cov for tests and mypy for type checking. Configuration is via .env (template in .env.example), and the app is run with: uvicorn src.main:app --reload.

Build system and dependency management

Build system and dependency management are Python-native. Dependencies are declared in pyproject.toml with separate dev groups, and uv is used to resolve, install, and lock them (lockfile: uv.lock). There is no separate build toolchain beyond standard Python packaging and uv commands like uv sync and uv add.

Implementation Status

Data Layer

Status	in progress
Assigned To	Ramesh
Completion percentage	90%
Blockers	GitHub account / environment issues

Python build / UV setup

Status	complete
Assigned To	Joe
Completion percentage	100%
Blockers	none

Wide Events (Logging)

Status	complete
Assigned To	Anthony
Completion percentage	100%
Blockers	none

Exception Middleware

Status	complete (handling for additional exceptions may be added)
Assigned To	Anthony
Completion percentage	99%
Blockers	none

README.md

Status	complete
Assigned To	Joe
Completion percentage	100%
Blockers	none

Pydantic models

Status	complete
Assigned To	Matt
Completion percentage	100%
Blockers	none

Agent service

Status	In progress
Assigned To	Matt
Completion percentage	90%
Blockers	delay in data layer availability

Project / Persona services

Status	complete
Assigned To	Matt
Completion percentage	100%
Blockers	none

Message service

Status	Not started
Assigned To	Matt
Completion percentage	0%
Blockers	delay in availability of data layer

Llama LLM adapter

Status	complete
Assigned To	Matt
Completion percentage	100%
Blockers	none

FastAPI controller w/ Neon auth integration

Status	In progress
Assigned To	Matt -> Anthony
Completion percentage	30%
Blockers	misunderstanding with implementation details, Neon Auth complication

Testing & Quality

Unit test coverage

Service and data layers have 100% coverage including some edge cases.

Integration test status

Decent integration testing in place for the data layer. Once full backend functionality is fully implemented with the message service, postman calls at the service API will be written and added to the CI/CD pipeline.

Code review process

The code review process is working well. We have pull requests set up to analyze code with CodeRabbitAI and then require 1+ approvals for merging to main. We also held a code review meeting Friday evening where team members' contributions were reviewed and needed changes were discussed.

Known bugs/issues and plans to address

This first implementation week was chaotic but productive. A small complication with the FASTAPI dependency injection of Neon auth delayed implementation of the API layer, Anthony is researching the correct way to implement, this will be resolved/implemented extremely early next week (assuming it hasn't been already).

The delay with the data layer becoming available blocked development of the message service and its integration into the agent service. This should also be implemented extremely early next week (assuming it hasn't been already).

Velocity & Week 6 Planning

Velocity calculation

We completed about 85% of the planned implementation but also got slightly ahead on frontend development planned for next week. Overall, we did complete the amount of work we estimated/planned, but some of it ended up being work that wasn't planned.

Lessons learned

We expected the Neon auth integration to be more straightforward than it turned out to be and one teammate had issues setting up their git environment, which I believe were ultimately straightened out

Week 6 plan

The API controller and Data layer should be completed very early in week 6 (or the end of week 5) so that the message service can be implemented and integrated into the agent service. The only other outstanding piece of the backend service is the agent sentiment service which will be implemented in week 6.

Our Monday evening meeting will start with the status of the 2 carryover items from week 5 detailed above and Anthony's presentation of the foundational work he already completed on the TanStack frontend application. We will discuss our divide and conquer strategy for the remaining work at that time as well.

We also need to arrange our deployment to AWS and set up the CI/CD pipeline. A team member was endeavoring to get the linting with ruff added as the first pipeline activity at the end of week 5.

Scope changes

We expect the delays in the 2 components identified above to be resolved by meeting time at 6pm Monday. If this holds true, there won't be any carryover change of scope.

Depending on how its development goes, the sentiment service could be scoped out of the MVP; we expect it to be able to be implemented without issue at this time.